

DOCUMENTO EXPLICATIVO DO PROJETO POKÉMON

Programação Orientada a Objetos em Java – AV1

Baseado no código-fonte fornecido e no enunciado da atividade.

1. Objetivo do projeto

O projeto implementa um sistema de batalhas inspirado no universo Pokémon, desenvolvido em Java com aplicação de conceitos de Programação Orientada a Objetos (POO). O programa permite escolher um Pokémon inicial, enfrentar batalhas em sequência, utilizar ataques normais ou especiais, defender-se, ganhar experiência, evoluir e enfrentar chefões.

O enunciado da AV1 determina a utilização integrada de variáveis e tipos primitivos, Scanner, classes e objetos, encapsulamento, herança e polimorfismo.

2. Estrutura geral das classes

O código possui uma superclasse chamada Pokemon e duas subclasses: PokemonInicial e PokemonLendario. Além dessas classes, o método main concentra a execução do jogo e utiliza `ArrayList<Pokemon>` para organizar grupos de Pokémon.

A atividade exige uma hierarquia com uma superclasse e pelo menos duas subclasses utilizando `extends` e `super(...)`. O projeto apresenta exatamente essa estrutura por meio de `Pokemon`, `PokemonInicial` e `PokemonLendario`.

3. Classe Pokemon – superclasse

A classe `Pokemon` é a base do sistema. Ela concentra características comuns aos Pokémon, como nome, vida, vida atual, ataque, ataque especial, velocidade, tipo, fraqueza, eficácia e experiência.

Entre os atributos, `nome`, `vida` e `vidaAtual` são declarados como `private`, enquanto `ataque`, `ataqueEspecial`, `velocidade` e `exp` são `protected`. `Tipo`, `fraqueza` e `eficácia` possuem acesso padrão. A declaração desses atributos pode ser observada no início da classe

4. Encapsulamento

O encapsulamento aparece principalmente nos atributos `private` e nos métodos `getters` e `setters`. O atributo `nome`, por exemplo, é acessado por `getNome()` e alterado por `setNome()`. O setter verifica se o `nome` é nulo ou vazio antes de aceitar o valor.

O atributo `vida` também possui `getter` e `setter`. O `setVida()` impede que a vida seja definida com valor menor ou igual a zero e ainda ajusta a `vidaAtual` caso ela fique maior que a nova vida. O `setVidaAtual()` impede que a vida atual fique abaixo de zero ou acima da vida máxima.

Essas validações representam invariantes: o objeto não aceita estados considerados inválidos. Isso atende diretamente ao objetivo da atividade de proteger o estado dos objetos usando private, getters, setters e validações.

5. Construtor e criação dos objetos

O construtor Pokemon recebe os principais dados de um Pokémon e utiliza os setters para inicializar nome, vida e vidaAtual. Os demais atributos são preenchidos diretamente. A experiência começa com 0.

No método main são criados diversos objetos Pokemon, PokemonInicial e PokemonLendario, cada um com seus atributos próprios. Exemplos incluem Pikachu, Raichu, Charmander, Bulbasaur, Squirtle, Darkrai, Mewtwo e Arceus.

6. Herança

A herança é utilizada quando PokemonInicial e PokemonLendario estendem Pokemon. Dessa forma, as subclasses herdam atributos e comportamentos comuns da superclasse.

PokemonInicial utiliza extends Pokemon e chama super(...) no construtor, reaproveitando a inicialização da superclasse. PokemonLendario segue a mesma ideia.

Essa estrutura representa uma relação 'é um': um Pokémon inicial é um Pokémon e um Pokémon lendário também é um Pokémon. O enunciado solicita exatamente uma hierarquia desse tipo com extends e super(...).

7. Uso de Protected

Os atributos ataque, ataqueEspecial e velocidade são protected na classe Pokemon. Isso permite que classes derivadas tenham acesso direto a esses dados quando necessário. Um exemplo é PokemonInicial, que altera ataque e ataqueEspecial nos métodos aumentarAtaque().

O uso de protected deve ser explicado no relatório como uma escolha para permitir que as subclasses especializadas modifiquem características de combate herdadas, enquanto os dados privados continuam protegidos pelos getters/setters. A atividade exige pelo menos um atributo protected com justificativa.

8. Polimorfismo e sobrescrita

O polimorfismo aparece por meio do método mostrarCategoria(). Na classe Pokemon ele informa que o objeto é um Pokémon. PokemonInicial sobrescreve esse método para informar que o objeto é um Pokémon inicial, enquanto PokemonLendario sobrescreve para informar que é um Pokémon lendário.

As subclasses utilizam @Override, demonstrando sobrescrita de método.

Importante: embora exista sobrescrita no código, a coleção principal usada nas batalhas contém objetos tratados como Pokemon. Para demonstrar de forma mais evidente o requisito da atividade, seria interessante utilizar explicitamente uma coleção polimórfica e percorrê-la com for-each chamando mostrarCategoria(). O enunciado pede essa chamada polimórfica em uma coleção.

9. Sobrecarga (overload)

A sobrecarga aparece em `PokemonInicial` nos métodos `aumentarAtaque()`. Existem duas versões com assinaturas diferentes: uma recebe apenas um inteiro e aumenta ataque e `ataqueEspecial` pelo mesmo valor; outra recebe dois inteiros, permitindo definir aumentos diferentes para ataque e `ataqueEspecial`.

Exemplos no código: `aumentarAtaque(int aumento)` e `aumentarAtaque(int aumentoAtaque, int aumentoEspecial)`. Isso caracteriza overload porque o mesmo nome de método possui parâmetros diferentes. O requisito de sobrecarga está previsto na atividade.

10. instanceof e downcasting

O método `evoluir()` verifica se o objeto atual é uma instância de `PokemonInicial` usando `instanceof`. Depois, realiza downcasting para `PokemonInicial`, permitindo acessar os métodos específicos da subclasse.

O trecho segue a lógica: `if (this instanceof PokemonInicial)` e, em seguida, `PokemonInicial inicial = (PokemonInicial) this`.

Isso atende ao requisito da atividade que solicita o uso de `instanceof` e downcasting com justificativa. Neste projeto, a justificativa é permitir que o método de evolução utilize os recursos específicos de `PokemonInicial` para aumentar os atributos.

11. Evolução dos Pokémon

A evolução é controlada pelo atributo `exp`. Ao vencer uma batalha, o Pokémon escolhido recebe 40 pontos de experiência. O método `evoluir()` verifica o nome e a quantidade de experiência e, quando os requisitos são atingidos, altera o nome, a vida, a velocidade e os atributos de ataque.

Por exemplo, Charmander com pelo menos 100 de experiência evolui para Charmeleon; com pelo menos 250 de experiência e estando como Charmeleon, evolui para Charizard. O mesmo conceito é aplicado a outras linhas evolutivas como Bulbasaur, Squirtle, Cyndaquil, Chikorita, Totodile, Torchic, Treecko e Mudkip.

12. Sistema de batalha

O método `batalha()` controla os turnos. Em cada turno, são mostradas as informações dos dois Pokémon e o jogador escolhe entre atacar e defender.

A velocidade determina quem age primeiro. Quando o jogador é mais rápido, ele executa sua ação antes do inimigo; caso contrário, o inimigo inicia o turno. O sistema também verifica se algum Pokémon foi derrotado.

A lógica de batalha utiliza ataque normal, ataque especial e defesa. O ataque especial recebe multiplicador de 1,30 quando é considerado eficaz e de 0,70 quando não é eficaz. A defesa reduz o dano recebido em 10 pontos.

13. Scanner e validação das entradas

O programa utiliza um Scanner estático chamado `teclado` para receber as escolhas do usuário pelo console. As entradas são validadas por estruturas `while`, impedindo opções fora dos valores esperados.

Por exemplo, na batalha, o usuário precisa escolher 1 para atacar ou 2 para defender; valores diferentes fazem o programa solicitar uma nova entrada. O mesmo padrão é utilizado na seleção de gerações e Pokémon.

A atividade exige um menu interativo com Scanner, validação de entrada e tratamento do conflito `nextInt()/nextLine()`. O código utiliza principalmente `nextInt()`, portanto o requisito específico de conflito `nextInt()/nextLine()` não fica demonstrado neste trecho e deve ser verificado antes da entrega.

14. Coleções ArrayList

O programa cria quatro `ArrayList<Pokemon>`. Elas são utilizadas para separar Pokémon iniciais, evoluções, formas finais e lendários. O método `Collections.addAll()` facilita o preenchimento das listas.

Essa organização permite selecionar inimigos aleatoriamente durante as batalhas e estabelecer diferentes mundos e chefões.

15. Fluxo do jogo

1. O jogador inicia o programa.
2. Escolhe uma geração entre as opções disponíveis.
3. Escolhe um Pokémon inicial.
4. Visualiza as informações do Pokémon e confirma a escolha.
5. Entra no Mundo 1 e enfrenta Pokémon escolhidos aleatoriamente.
6. A cada vitória, recebe experiência e pode evoluir.
7. Após determinadas batalhas, aparecem novos mundos e chefões.
8. O jogador pode continuar ou parar após uma batalha.
9. Se perder, pode escolher recomeçar.
10. Ao derrotar Arceus após as batalhas previstas, o jogo informa que foi concluído.

A seleção de Pokémon e a organização das batalhas aparecem principalmente no método `main()`, incluindo as listas de Pokémon, seleção de geração, escolha do inicial e progressão das batalhas.

16. Pontos que merecem atenção antes da entrega

Verificar a exigência de no mínimo 5 classes próprias além da `Main/App`, pois o código apresentado possui 3 classes próprias claramente identificadas.

Revisar o requisito de todos os atributos serem `private` com `getters/setters`. O código possui atributos `protected` e atributos com acesso padrão.

Adicionar, se necessário, uma demonstração explícita de coleção polimórfica percorrida com `for-each` chamando o método sobrescrito.

Verificar o requisito de tratamento de `nextInt()/nextLine()`, pois ele não aparece de forma evidente no código fornecido.

Confirmar a compilação e execução pelo terminal ou IDE antes da entrega.

Preparar uma explicação oral de cada conceito, porque o enunciado informa que cada aluno poderá ser questionado sobre qualquer parte do código

17. Conclusão

O projeto apresenta uma aplicação prática de POO em Java usando o tema Pokémon. Os principais conceitos aparecem na estrutura da classe `Pokemon`, na herança das classes especializadas, na sobrescrita de métodos, na sobrecarga, no uso de `instanceof/downcasting`, no encapsulamento de atributos, no sistema de batalhas e no uso de coleções.

Ao mesmo tempo, a comparação com o enunciado mostra alguns requisitos que precisam ser conferidos ou complementados, principalmente a quantidade mínima de classes, a regra de todos os atributos `private`, a demonstração explícita de polimorfismo em coleção e o tratamento de `nextInt()/nextLine()`. O objetivo desta análise é servir como base para a apresentação e para o relatório individual, sem afirmar como atendido aquilo que não está demonstrado no código.

18. Referências utilizadas

- Código-fonte fornecido: classe `Pokemon` e classes derivadas `PokemonInicial` e `PokemonLendario`.
- Projeto Avaliativo 1 (AV1) – Programação Orientada a Objetos em Java, UniAnchieta, Prof. Jesse Guimarães.

RELATÓRIO INDIVIDUAL – PROJETO POKÉMON

1. Identificação

Aluno: Murillo Oliveira | **Curso:** Engenharia de Software | **Disciplina:** Programação Orientada a Objetos

Professor: Jesse Guimarães | **Instituição:** UniAnchieta | **Projeto:** Sistema de Batalhas Pokémon – AV1

2. Minha participação no projeto

Particpei das discussões sobre as opções oferecidas ao jogador e sugeri Pokémon, ataques, evoluções, mundos e chefões. Acompanhei o desenvolvimento para compreender como as ideias eram transformadas em funcionalidades Java. Também ajudei a identificar possíveis melhorias e problemas, além de discutir os requisitos da atividade e os conceitos de POO que deveriam constar no projeto. Minha participação direta na programação foi menor que a de Gabriel.

Encapsulamento: a classe Pokemon declara vida como private e usa getVida() e setVida(). Essa decisão controla alterações no atributo e impede que a vida máxima receba um valor menor ou igual a zero. Acompanhei a discussão dessa solução, embora não tenha sido o principal responsável por implementá-la.

Herança: PokemonInicial e PokemonLendario estendem Pokemon. Essa organização reaproveita os atributos e comportamentos comuns da classe principal e permite especializar os tipos de Pokémon sem repetir toda a estrutura.

3. Dificuldades encontradas

Uma dificuldade foi transformar ideias para o jogo em funcionalidades viáveis em Java. Ao acompanhar o projeto, passei a compreender melhor classes, objetos, encapsulamento, herança, polimorfismo, sobrescrita, sobrecarga, instanceof e downcasting.

4. O que eu aprendi

Ao acompanhar o desenvolvimento, compreendi melhor como classes e objetos representam Pokémon, como getters e setters validam dados e como a herança reaproveita características comuns. Também aprendi que planejar funcionalidades, testar e discutir problemas são partes importantes do projeto.

Conclusão e referências

Contribuí principalmente com ideias, sugestões, discussões e acompanhamento da implementação. O trabalho em equipe ajudou a relacionar os conceitos estudados com um sistema prático. Referências: código-fonte do Projeto Pokémon – AV1; enunciado da AV1; materiais apresentados nas aulas de Programação Orientada a Objetos.

RELATÓRIO INDIVIDUAL – PROJETO POKÉMON

1. Identificação

Aluno: Gabriel F Stringuetti | **Curso:** Engenharia de Software | **Disciplina:** Programação Orientada a Objetos

Professor: Jesse Guimarães | **Instituição:** UniAnchieta | **Projeto:** Sistema de Batalhas Pokémon – AV1

2. Minha participação no projeto

Criei a estrutura da classe Pokemon e das classes especializadas PokemonInicial e PokemonLendario. Desenvolvi a lógica de batalhas, incluindo turnos, velocidade, ataques normais e especiais, cálculo de dano, defesa e derrota dos Pokémon. Implementei também experiência e evolução após vitórias, organizei grupos de Pokémon com ArrayList e utilizei Scanner para as escolhas feitas no console. Durante o desenvolvimento, testei e ajustei o código para integrar essas partes.

Encapsulamento: mantive nome, vida e vidaAtual como atributos private na classe Pokemon e utilizei getters e setters. O setVida() rejeita valores menores ou iguais a zero; o setVidaAtual() limita a vida atual ao intervalo entre zero e a vida máxima. Assim, as alterações nesses dados passam por validações.

Herança: criei PokemonInicial e PokemonLendario como subclasses de Pokemon, utilizando extends e super(...) para aproveitar a inicialização e os comportamentos comuns. Essa estrutura evita repetir a base de cada Pokémon e permite especializações; os métodos sobrescritos com @Override demonstram polimorfismo.

3. Dificuldades encontradas

As maiores dificuldades foram organizar as responsabilidades das classes, acertar a sequência de turnos, aplicar regras de ataque, defesa, velocidade e dano e controlar os requisitos de evolução de cada Pokémon. Também precisei revisar o código à luz dos requisitos da atividade, o acesso aos atributos e a demonstração do polimorfismo.

4. O que eu aprendi

Aprendi a aplicar construtores, encapsulamento, herança, sobrescrita, sobrecarga, instanceof e downcasting em um programa maior. Reforcei o uso de ArrayList, Scanner, condições, repetições e testes. O projeto mostrou como as decisões de estrutura influenciam o funcionamento do sistema.

Conclusão e referências

Fui o principal responsável pela implementação, desde a estrutura inicial até batalhas, evolução e seleção dos Pokémon. Os testes e a revisão dos requisitos contribuíram para meu aprendizado. Referências: código-fonte do Projeto Pokémon – AV1; enunciado da AV1; materiais apresentados nas aulas de Programação Orientada a Objetos.

RELATÓRIO INDIVIDUAL – PROJETO POKÉMON

1. Identificação

Aluno: Victor H Bissoli | **Curso:** Engenharia de Software | **Disciplina:** Programação Orientada a Objetos

Professor: Jesse Guimarães | **Instituição:** UniAnchieta | **Projeto:** Sistema de Batalhas Pokémon – AV1

2. Minha participação no projeto

Minha principal contribuição foi trabalhar na base da estrutura de cada Pokémon. Participei da organização das características que cada objeto precisa receber, como nome, tipo, vida, ataque, ataque especial, velocidade, fraqueza e eficácia. Essas informações diferenciam os Pokémon e influenciam sua participação nas batalhas.

Ao pensar nessa estrutura, considerei quais dados são comuns a todos os Pokémon e quais valores mudam de um personagem para outro. A classe Pokemon concentra os atributos comuns, e o construtor recebe os valores necessários para criar cada objeto. Isso facilitou a definição de Pokémon diferentes sem criar uma classe independente para cada um.

Uma decisão de encapsulamento presente nessa base é manter a vida como atributo private e usar métodos como getVida() e setVida() para acessar ou alterar seu valor. Assim, o programa pode impedir uma vida máxima inválida. Na herança, PokemonInicial e PokemonLendario estendem Pokemon, reaproveitando os dados e comportamentos comuns. Meu foco foi a estrutura dos Pokémon, em conjunto com o desenvolvimento geral da equipe.

3. Dificuldades encontradas

A principal dificuldade foi definir características que servissem a todos os Pokémon sem deixar de representar as diferenças entre eles. Também foi necessário pensar em valores coerentes para que vida, ataques e velocidade tivessem efeito no sistema de batalha.

4. O que eu aprendi

Aprendi como classes e objetos representam personagens de um jogo, como o construtor inicializa seus atributos e por que uma estrutura comum facilita adicionar novos Pokémon. Entendi melhor o encapsulamento e o uso da herança para especializar tipos de objetos.

Conclusão e referências

Contribuí para a organização da base dos Pokémon e para a definição de seus atributos. Essa etapa ajudou a conectar a estrutura dos objetos às demais funcionalidades do jogo. Referências: código-fonte do Projeto Pokémon – AV1; enunciado da AV1; materiais apresentados nas aulas de Programação Orientada a Objetos.

RELATÓRIO INDIVIDUAL – PROJETO POKÉMON

1. Identificação

Aluno: João Gabriel | **Curso:** Engenharia de Software | **Disciplina:** Programação Orientada a Objetos

Professor: Jesse Guimarães | **Instituição:** UniAnchieta | **Projeto:** Sistema de Batalhas Pokémon – AV1

2. Minha participação no projeto

Minha participação foi ajudar a pensar no sistema de batalha. Contribuí com ideias sobre as opções de ataque e defesa, a ordem dos turnos e as condições para determinar o fim de um confronto. O objetivo era que as ações escolhidas pelo jogador produzissem resultados claros durante o jogo.

Discuti como a velocidade dos Pokémon poderia influenciar quem age primeiro e como ataques normais e especiais poderiam causar danos diferentes. Também acompanhei a definição de regras de defesa, vitória e derrota. Essas decisões serviram de base para a lógica implementada no código.

Uma decisão de encapsulamento importante para a batalha é controlar `vidaAtual` por métodos, impedindo valores abaixo de zero ou acima da vida máxima. Isso ajuda a apresentar o estado do Pokémon de forma consistente. Quanto à herança, os Pokémon iniciais e lendários usam a base da classe `Pokemon`, de modo que as batalhas possam trabalhar com características comuns, como vida, ataque e velocidade.

3. Dificuldades encontradas

A dificuldade principal foi pensar em regras que funcionassem em situações diferentes: Pokémon mais rápido ou mais lento, ataque ou defesa e término da batalha após a derrota de um participante. Cada regra precisava fazer sentido em conjunto com as demais.

4. O que eu aprendi

Aprendi a planejar a lógica de um sistema antes de implementá-la, identificando entradas do jogador, ordem das ações e condições para encerrar uma batalha. Também compreendi como os atributos dos objetos e os conceitos de POO influenciam o funcionamento do combate.

Conclusão e referências

Contribuí com ideias e discussões para as regras de batalha. O projeto me ajudou a entender a relação entre planejar o comportamento do jogo e transformá-lo em métodos e condições no programa. Referências: código-fonte do Projeto Pokémon – AV1; enunciado da AV1; materiais apresentados nas aulas de Programação Orientada a Objetos.